

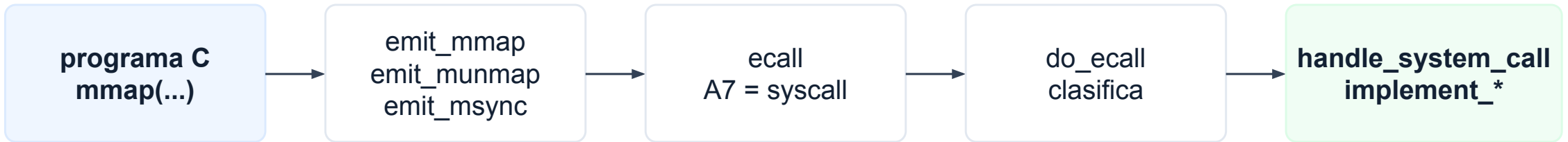
Memory Mappings en Selfie

Noe Paredes Rauraico
Yofer Mauricio Corne Flores



Primero conectamos las syscalls al kernel

Antes de implementar la semántica, necesitábamos que Selfie reconociera las llamadas nuevas.

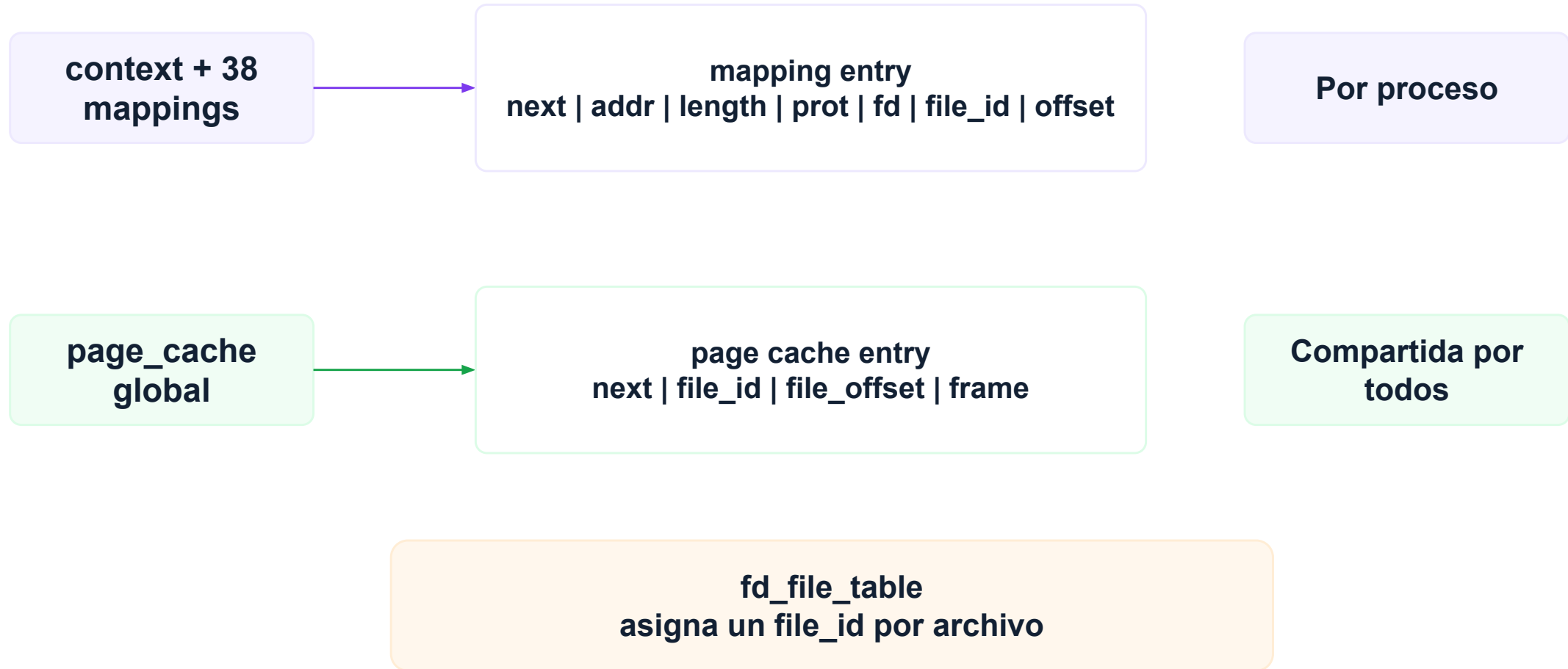


A0 = addr
A1 = length
A2 = prot
A3 = fd
A4 = offset
A7 = SYSCALL_MMAP

El kernel termina llamando a:
`implement_mmap /`
`implement_munmap /`
`implement_msync`

Después agregamos las estructuras que faltaban

La solución necesitaba recordar “qué región pertenece a qué archivo” y compartir páginas por archivo + offset.



mmap: crear una región virtual respaldada por archivo



```
while (i < pages) {  
    virtual_page = page(addr + i * PAGE_SIZE);  
    file_page_offset = offset + i * PAGE_SIZE;  
  
    frame = get_or_create_cache_frame(file_id, file_page_offset, fd);  
    map_mmap_page(context, virtual_page, frame);  
}
```

Después de mmap, la page table ya apunta a frames válidos.

Page cache - Comparte memoria

La clave no es la dirección virtual, sino el par:

(file_id, file_offset)

get_or_create_cache_frame(file_id, file_offset,
fd)

find_page_cache_entry(...)

read_file_page_into_frame primero limpia
la página con ceros y luego lee hasta
PAGESIZE.

HIT
reusar el mismo frame

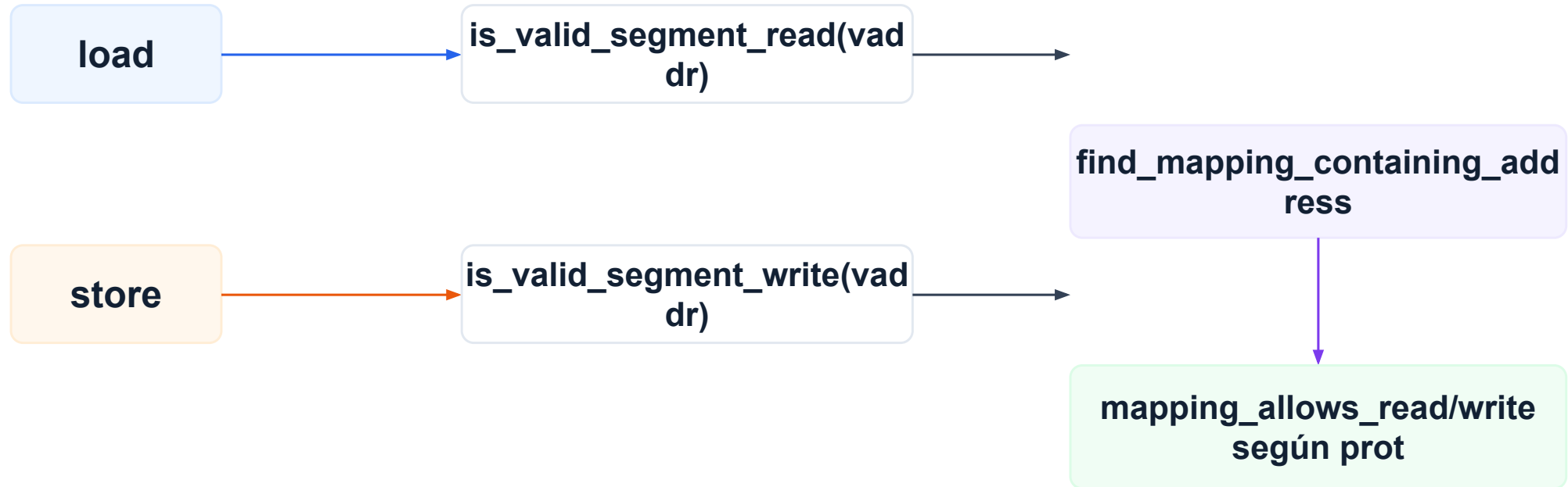
dos mappings del mismo archivo +
offset
ven el mismo contenido

MISS
palloc + leer archivo +
registrar entrada

si el archivo es pequeño,
lo restante queda en cero

Los permisos se validan cuando el programa accede

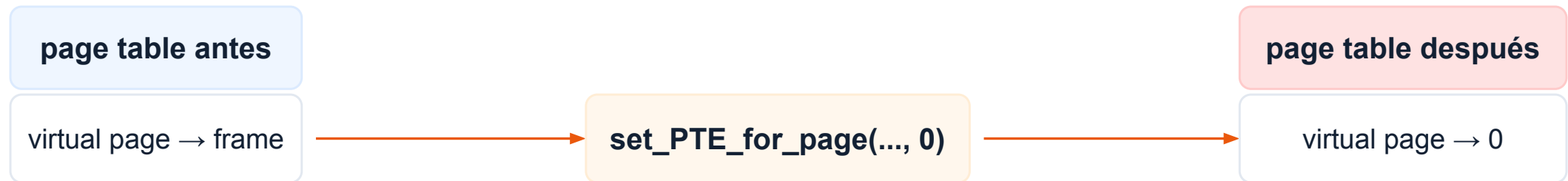
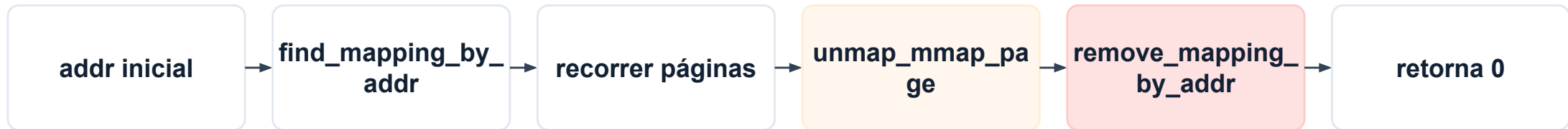
mmap crea la región; los accesos reales pasan por la lógica existente de load/store.



Si no cumple: no se trata como página válida del segmento; termina en error de acceso.

munmap: desmontar lo que mmap creó

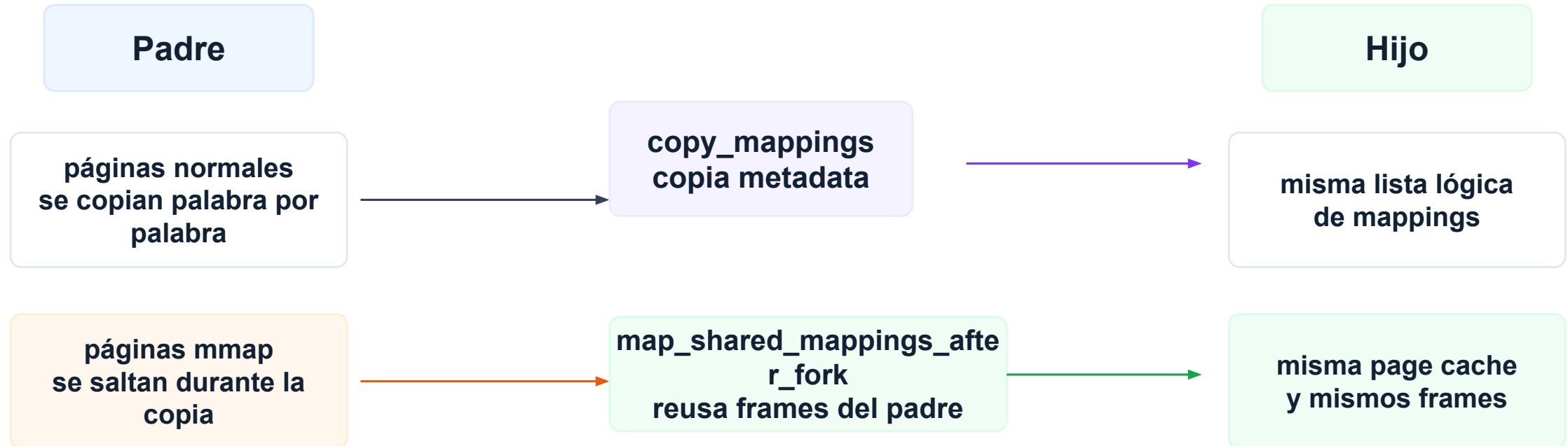
munmap no escribe al archivo. Solo elimina la relación virtual del proceso.



La page cache no se borra, ya que puede seguir siendo útil si otro mapping usa el mismo frame.

fork: copiar el proceso sin duplicar mappings

El reto: las páginas normales se copian, pero las páginas mmap deben seguir compartiendo frames.



Así se mantiene la semántica: si el padre o el hijo escriben en un mapping compartido, ambos apuntan al mismo frame.

msync: escribir los frames de vuelta al archivo

msync convierte cambios en memoria en cambios visibles dentro del archivo.



```
file_offset = mapping.offset + i * PAGE_SIZE  
frame = get_frame_for_page(page_table, page)  
write_frame_to_file_page(fd, file_offset, frame)
```

**El offset original del mapping es fundamental:
permite escribir cada frame en la posición correcta del archivo.**

Éxito: A0 = 0 Error: A0 = -1

Qué cubren los tests que validan la implementación

mmap básico

leer archivo, offsets, varias páginas, redondeo

page cache

dos mappings reutilizan frame compartido

msync

writes en memoria llegan al archivo

munmap

después de desmontar, la región deja de ser válida

fork

padre e hijo conservan mappings compartidos

errores

fd inválido, prot inválido, length cero, alineación